

01/23

Databases

Databases is the foundational of almost every software, Here is why it matters.

① Central source of truth: DB stores all business data - orders, users, transactions, analytics, logs etc..

② Support every Feature:

Every API call, UI screen, business report and integration depends on the shape

③ Avoids future problems:

Poor early decision can lead to data corruption, slow performance, bottlenecks issues.

④ Enables scaling:

A good DB design lets you grow from 1000 to million users and change new features without massive rewrites.

II Fundamental responsibilities of DB system

1] Data organization:

- (i) Store information in structured way.
- (ii) Supports relationships b/w records.

2] Data Integrity:

- (i) Enforce rules for accuracy
- (ii) Support concurrent access.

3] Access [CRUD]

- (i) Provide reliable interfaces to CRUD
- (ii) Support concurrent access and transaction isolation, so that multiple users can safely interact with the data.

4] Security:

- (i) Manage authentication (who can access)
& authorization.

- (ii) Support role based access control, encryption

5] Performance & Scalability.

- (-) Ensure data can be retrieved, modified and stored efficiently.
- (-) Use indexes, caching, partitioning and replication to support high loads and scalability.

6] Availability & Durability:

- (-) Protect data against loss
- (-) Guarantee availability and recovery from failures



Relational

- (tables, rows, cols)
- predefined schema
- Data Integrity
- ⇒ SQL

Non-Relational

- MongoDB
- No strict schema
- 2 same collections can exist

01/10 Migrations

DB migration refer to the proces of changing dB structure (schema), moving data or upgrading schema in a controlled, repeatable way. They are essential for evolving systems and maintain consistency.

→ Why do we need it?

① Evolving requirements: As app grows, requirements increase.

② Consistency: Teams and env stay in sync because every migration is versioned and tracked.

③ Deployment: Easily lets you roll with new features

④ Data movement: Migrating b/w DB engines cloud to on-prem

⑤ Compliance / Best practice: Reduce manual errors, prevent data loss

11/06

Up and Down migrations

✓

→ Up Migration is the script that applies a change to your DB schema.

→ It moves your schema forward :
eg: Creating a new table, add column.

→ Always written in the direction of new features

Down migration : Reverses the changes made by corresponding up migration.
It reverts the schema.

Trigger : Whenever a special DB Command happens in a specified event (like update, insert, update, DELETE)

(*) why would we use Triggers?

→ ① Reliability :

Trigger enforces rules directly in the DB, so updates happen - even if developer forgets or backed changes.

② Consistency :

→ In large apps, many services and backgrounds might update the DB.

→ By putting the rule in the DB, you guarantee every update from every API, admin tool or batch job gets the right stamp.

③ Maintenance & Code simplification :

(i) Devs don't remember to update the updated - at everywhere.

(ii) Less repeated code in controller / service - reducing bugs.

④ Auditing / Compliance : Easier to trace down bugs.

③ Future proofing :

Better for future, triggers keeps business rules consistent.

(*) Indexing

⇒ Indexing is like an index in a book, it helps you find things quickly.

→ When search for a value in a large DB, DB can use index to jump straight the way we want.

→ (*) Why do we need indexes?

⇒ Backend API usually fetch by ID, email, product name or other field.

→ Without Indexes, DB will be slow

→ With Indexes, lookups are instant

(*) Generally, indexes on foreign keys speed up JOINS and queries to get all Orders

(*) Impact on large DBs :

→ For read-heavy APZs, (millions) indexes are critical, queries without indexes get extremely slow as tables grow.

(Q) How do Indexes affect API calls?

→ READ (SELECT) : Much faster with Indexes.

→ Writes (insert/update/DELETE) : Much slower

→ Disk Space : Every Index uses storage.

→ Too many Indexes : Bloats space, slow work.